



Práctica 7

Organización y Arquitectura de Computadoras

Docente: Garcia Rocha Jose Isabel

Alumno:

- Chaparro Herrera Hugo Giovanni - 2200357

Tijuana, Baja California a 16 de octubre de 2025



Objetivo

Seleccionar el modo de direccionamiento adecuado, para manejo de memoria en aplicaciones de sistemas basados en microprocesador, mediante el análisis de los diversos modos de direccionamiento existentes, de forma responsable y eficiente.

Dificultades durante el desarrollo

Durante el desarrollo de la práctica, la única dificultad que se presentó fue tener que comprender correctamente cuáles eran los registros necesarios para mandar a la pila para así no afectar al flujo del programa al entrar en una subrutina.

Desarrollo

- Implementar una subrutina que cuente el número de veces que aparece un carácter en una cadena del '0' al '9' utilizar una tabla en memoria y xlat para mostrar el número con putchar.

Se captura la cadena usando la subrutina de capturar y el carácter a buscar usando la rutina getche y se imprimen junto a sus respectivos prompts.

```
; imprimir prompt para capturar cadena
mov edx, cad_prompt
call newputs
call salto

mov ebx, cad          ; se manda la direccion de la cadena a ebx
call capturar         ; se llama al procedimiento de capturar la cadena

call salto
mov edx, cad_ingresada
call newputs          ; se imprime el prompt de cadena capturada
call salto

; imprime la cadena
mov edx, cad
call newputs
call salto
```

```
koria@Giolaptop:/mnt/c/Users/hugog/OneDrive/Documentos/6to Semestre/OAC/Practica7$ ./Practica7
Ingresa una cadena: (que termine en *):
palabra*
Cadena ingresada:
palabra
```

```
; se imprime el prompt de ingresar caracter
mov edx, char_prompt
call newputs
call salto
```



```
; se captura el caracter
mov eax, 0
call getche
mov ebx, caracter
mov [ebx], al
call salto

; se imprime el prompt del caracter ingresado
mov edx, char_ingresada
call newputs
call salto

; se imprime el caracter
mov ebx, caracter
mov al, [ebx]
call putchar
call salto
```

```
Ingresa un caracter a buscar:
a
Caracter ingresado:
a
```



Utilizando una tabla declarada en .data, realizamos una subrutina que proceda a contar el número de veces que al (carácter en la cadena a comparar) es igual a edx (carácter buscado).

Cada vez que se encuentra este carácter, se va sumando 1 a cl, que lo estamos usando como contador de ocurrencias. Para una vez recorrida toda la cadena, mandamos el valor de cl a al y a ebx la dirección de la tabla, con la finalidad de usar XLAT para mandar el número de veces que apareció.

```
contar:
    push eax
    push ecx
    mov eax, 0                ; se reinicia eax en cero para resetear AL y
usarlo como contador
    mov cl, 0                 ; registro que usaremos como contador
    ciclo_contar:
        mov al, [ebx + edi]    ; se manda el caracter a comparar a AL
        cmp al, '%'           ; se verifica si ya se llego al final de la cadena
        je fin_contar         ; si se llega al final, se sale del bucle

        cmp al, [edx]          ; se compara AL con el caracter a buscar
        je sumar              ; de ser cierto, se le suma 1 al contador de
apariciones

        inc edi                ; se incrementa EDI para dirigirse al siguiente
caracter
        jmp ciclo_contar      ; se continua el bucle
    sumar:
        inc cl                 ; se incrementa CL en 1 para acumular apariciones
        inc edi                ; se incrementa EDI en 1 para dirigirse al
siguiente caracter
        jmp ciclo_contar
    fin_contar:
        mov edx, contar_prompt ; se imprime el prompt
        call newputs           ; se manda a llamar la subrutina para imprimir
        call salto             ; se imprime un salto de línea

        mov ebx, tabla         ; se manda la direccion de la tabla a EDX
        mov al, cl              ; se copia el contador a CL
        xlat                   ; se extrae y guarda el numero de apariciones en
AL
        call putchar           ; imprime el numero de iteraciones
        call salto

    pop eax
    pop ecx
    ret
```

```
El caracter ingresado aparece en la cadena:
3
```



Código

```
%include "../Libreria/pc_io.inc" ; se incluye la libreria

section .text
    global _start:

_start:
    ; ===== CAPTURAR CADENA Y CONTAR LAS VECES QUE APARECE UN CARACTER
    ; imprimir prompt para capturar cadena
    mov edx, cad_prompt
    call newputs
    call salto

    mov ebx, cad ; se manda la direccion de la cadena a ebx
    call capturar ; se llama al procedimiento de capturar la cadena

    call salto
    mov edx, cad_ingresada ; se manda el contenido a ebx con el prompt para poder ser
    impreso
    call newputs ; se imprime el prompt de cadena capturada
    call salto

    ; imprime la cadena
    mov edx, cad
    call newputs
    call salto

    ; se imprime el prompt de ingresar caracter
    mov edx, char_prompt
    call newputs
    call salto

    ; se captura el caracter
    mov eax, 0
    call getche
    mov ebx, caracter
    mov [ebx], al
    call salto

    ; se imprime el prompt del caracter ingresado
    mov edx, char_ingresada
    call newputs
    call salto

    ; se imprime el caracter
    mov ebx, caracter
    mov al, [ebx]
    call putchar
    call salto

    ; ===== XLAT del 0 al 9
```



```
mov ebx, cad           ; se manda la direccion de la cadena a EBX
mov edx, caracter       ; se manda la direccion del caracter a EDX
call contar             ; se manda a llamar a la subrutina de contar caracter

; SYS_EXIT
mov eax, 1
mov ebx, 0
int 80h

; =====

capturar:
    pushad
    mov edi, 0
    ciclo_cap:
        call getche      ; se captura el caracter y se almacena el al
        cmp al, '*'      ; se compara con el indicador de fin de cadena
        je fin           ; si al == *, se sale del ciclo capturar

        cmp al, 13       ; si al == ENTER, no se captura
        je ciclo_cap

        cmp al, 10       ; si al == NUEVA_LINEA, no se captura
        je ciclo_cap

        mov [ebx + edi], al ; se coloca el caracter capturado en ebx + indice (edi)
        inc edi           ; se incrementa el indice
        jmp ciclo_cap     ; se reinicia el ciclo

    fin:
        mov al, '%'      ; se coloca el caracter '%' en al
        mov [ebx + edi], al ; se coloca el % al final de la cadena
    popad
    ret

contar:
    push eax
    push ecx
    mov eax, 0            ; se reinicia eax en cero para resetear AL y usarlo
                           ; como contador
    mov cl, 0             ; registro que usaremos como contador
    ciclo_contar:
        mov al, [ebx + edi] ; se manda el caracter a comparar a AL
        cmp al, '%'         ; se verifica si ya se llego al final de la cadena
        je fin_contar      ; si se llega al final, se sale del bucle

        cmp al, [edx]       ; se compara AL con el caracter a buscar
        je sumar           ; de ser cierto, se le suma 1 al contador de
                           ; apariciones
    apariciones
```



```
        inc edi                ; se incrementa EDI para dirigirse al siguiente
caracter
        jmp ciclo_contar      ; se continua el bucle
sumar:
        inc cl                ; se incrementa CL en 1 para acumular apariciones
        inc edi               ; se incrementa EDI en 1 para dirigirse al siguiente
caracter
        jmp ciclo_contar
fin_contar:
        mov edx, contar_prompt ; se imprime el prompt
        call newputs           ; se manda a llamar la subrutina para imprimir
        call salto             ; se imprime un salto de linea

        mov ebx, tabla         ; se manda la direccion de la tabla a EDX
        mov al, cl              ; se copia el contador a CL
        xlat                   ; se extrae y guarda el numero de apariciones en AL
        call putchar           ; imprime el numero de iteraciones
        call salto

    pop eax
    pop ecx
    ret

newputs:
    pushad
    mov esi, 0
prnt:
    mov al, [edx + esi]        ; se manda a al el caracter inicial
    cmp al, '%'                ; se compara al con %
    je finputs                 ; si es verdadero, se sale del ciclo

    call putchar               ; se imprime al caracter actual
    inc esi                    ; se incrementa el indice
    jmp prnt                    ; se reinicia el ciclo de impresion
finputs:
    popad
    ret

newputs2: ; recibe en EDX la direccion y en ESI el indice a mostrar
    pushad
    mov edi, 0
ciclo_newputs2:
    add edx, edi
    mov al, [edx + esi * 4]
    cmp al, '%'
    je fin_ciclo_newputs2

    inc edi
    call putchar
    jmp ciclo_newputs2
fin_ciclo_newputs2:
    popad
    ret
```



```
salto:
    pushad
    mov al, 13
    call putchar

    mov al, 10
    call putchar
    popad
    ret

section .data
    cad_prompt: db "Ingresa una cadena: (que termine en *):%"

    cad_ingresada: db "Cadena ingresada:%"
    char_ingresada: db "Caracter ingresado:%"

    char_prompt: db "Ingresa un caracter a buscar:%"
    contar_prompt: db "El caracter ingresado aparece en la cadena:%"
    tabla: db '0','1','2','3','4','5','6','7','8','9'

section .bss
    cad resb 256
    caracter resb 1

    cad_len resb 1

    numero resb 1
```

Referencias Bibliográficas

- [02. NASMx86: partes del código fuente](#)
- [Guide to x86 Assembly](#)
- [Using as - Assembler Directives](#)
- [NASM Assembly Language Tutorials - asmtutor.com](#)
- [Assembly - System Calls](#)